Spring Boot 数据库访问

准备工作：环境与基础配置

- 课前环境要求
- 数据库准备—建表 SQL
- POM.xml 依赖
- application.yml 配置
- 实体类 User
- 项目目录结构

课时 1：MyBatis 纯注解实现单表 CRUD

- 学习目标
- 为什么选择 MyBatis?
- MyBatis 核心注解
- 课堂练习：注解实现单表 CRUD
  - 第一步：编写 Mapper 层代码（UserMapper）
  - 第二步：编写 Service 层代码（UserService）
  - 第三步：编写 Controller 层代码（UserController）
  - 第四步：接口测试

课时 2：MyBatis XML + 动态SQL

- 学习目标
- 为什么需要 XML 映射?
- 课堂练习：MyBatis XML 方式实现动态 SQL
  - 第一步：配置 XML 映射文件路径
  - 第二步：UserMapper 新增方法
  - 第三步：编写 XML 映射文件（UserMapper.xml）
  - 第四步：编写 Service + Controller 层代码
  - 第五步：动态条件查询测试

课时 3：MyBatis-Plus 极速开发

- 学习目标
- 什么是 MyBatis-Plus?

课堂练习：MP 极速开发
第一步：引入 MyBatis-Plus 依赖
第二步：编写 MP 版 Mapper（UserMPMapper）
第三步：实现条件 + 分页查询（Service层）
第四步：编写分页查询接口（Controller 层）
第五步：配置 MP
第六步：MP 分页查询测试
课时 4：事务管理 + 项目分层规范
学习目标
什么是事务？为什么需要事务？
Spring Boot 声明式事务—@Transactional
课堂练习：带事务的业务方法
第一步：编写带事务的业务方法（UserTxService)
第二步：编写事务测试接口（UserTxController）
第三步：事务回滚验证
测试1：正常情况（无异常，事务提交）
测试2：异常情况（有异常，事务回滚）
企业级开发规范
1. 技术选择规范
2. 分层开发规范
3. 通用规范
课程总结 + 常见问题
核心知识点总结
常见问题答疑

准备工作：环境与基础配置

课前环境要求

1. 开发环境：JDK17+、Maven3.9+、IDEA2025+
2. 数据库：MySQL8.0+

3. 核心依赖：Spring Boot 3.5.11、MyBatis-starter 3.0.5、MyBatis-Plus-starter 3.5.7
4. 工具：Apifox（接口测试）、Navicat（数据库操作）

数据库准备—建表 SQL

> 说明：下划线命名法（适配驼峰转换）

```sql
CREATE DATABASE IF NOT EXISTS springboot3_db DEFAULT CHARACTER SET utf8mb4;

USE springboot3_db;

DROP TABLE IF EXISTS `t_user`;

CREATE TABLE `t_user` (
  id BIGINT AUTO_INCREMENT PRIMARY KEY COMMENT '主键',
  username VARCHAR(50) NOT NULL COMMENT '用户名',
  password VARCHAR(100) NOT NULL COMMENT '密码',
  age INT NULL COMMENT '年龄',
  email VARCHAR(100) NULL COMMENT '邮箱',
  create_time DATETIME DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='用户表';
```

POM.xml 依赖

> 说明：Spring Boot3 父依赖 + 必备启动器，版本统一适配

```xml
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.5.11</version>
</parent>

<properties>
  <!--统一管理依赖版本号-->
  <java.version>17</java.version>
  <mysql.version>8.3.0</mysql.version>
  <mybatis-starter.version>3.0.5</mybatis-starter.version>
</properties>

<!-- Web + MyBatis + MySQL驱动 + Lombok -->
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>


  <dependency>
    <groupId>org.mybatis.spring.boot</groupId>
    <artifactId>mybatis-spring-boot-starter</artifactId>
    <version>${mybatis-starter.version}</version>
    <scope>compile</scope>
  </dependency>


  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>${mysql.version}</version>
    <scope>compile</scope>
  </dependency>


  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>
</dependencies>
```

application.yml 配置

> 说明：数据源配置（MySQL8 专用驱动）+ MyBatis 驼峰转换

```yaml
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/springboot3_db?useUnicode=true&characterEncoding=utf8&serverTimezone=Asia/Shanghai
    username: root # 替换为自己的用户名
    password: 123456 # 替换为自己的密码
    driver-class-name: com.mysql.cj.jdbc.Driver

mybatis:
  configuration:
    map-underscore-to-camel-case: true # 下划线→驼峰自动映射
```

实体类 User

> 说明：Lombok @Data 简化代码 + LocalDateTime（JDK8+ 推荐时间类型）

```java
package top.mqxu.week05.entity;

import lombok.Data;
import java.time.LocalDateTime;

@Data
public class User {
    private Long id;
    private String username;
    private String password;
    private Integer age;
    private String email;
    private LocalDateTime createTime;
}
```

项目目录结构

> 说明：企业标准分层结构

```perl
top.mqxu.week05
├── SpringbootWeek05Application.java     // 启动类
├── entity/                              // 实体类层
├── mapper/                              // 数据访问层
├── service/                             // 业务逻辑层
└── controller/                          // 接口控制层
```

课时 1：MyBatis 纯注解实现单表 CRUD

学习目标

1. 掌握 Spring Boot3 整合 MyBatis 的核心流程
2. 会用 MyBatis 注解实现单表增/删/改/查
3. 理解 @Mapper、#{参数}、主键回显的核心作用
4. 能独立测试数据库操作接口

为什么选择 MyBatis?

- 企业主流：国内 80% 以上 Java 后端项目使用 MyBatis
- 核心优势：手写 SQL 可控、性能优、适配复杂业务
- 注解适用场景：**简单单表操作**（开发快、无冗余 XML）
- 官网：https://mybatis.org/mybatis-3/zh_CN/index.html

MyBatis 核心注解

注解	作用	适用场景

@Mapper	标记 Mapper 接口，生成代理对象	所有 Mapper 接口
@Insert	新增数据注解	单条/批量新增
@Select	查询数据注解	单条/列表查询
@Update	修改数据注解	单条/批量修改
@Delete	删除数据注解	单条/批量删除
@Options	配置主键回显等参数	新增操作（自增主键）

#{参数} 的优势

- 预编译 SQL：底层自动做参数转义，**从根本防止 SQL 注入**（企业必须用）
- 自动参数匹配：直接对应实体类字段/方法参数，无需手动封装
- 反例：${参数}（字符串拼接，有注入风险，仅特殊场景使用）

课堂练习：注解实现单表 CRUD

第一步：编写 Mapper 层代码（UserMapper）

> 说明：接口无需实现类，MyBatis 会自动生成代理对象，主键回显配置

```java
package top.mqxu.week05.mapper;

import org.apache.ibatis.annotations.*;
import top.mqxu.week05.entity.User;

import java.util.List;

/**
 * @author mqxu
 * @description 针对表【t_user(用户表)】的数据库操作 Mapper
 * @createDate 2026-04-02 15:10:24
 */
@Mapper
public interface UserMapper {

    /**
     * 插入数据
     *
     * @param record 插入的数据记录
     * @return 受影响的记录行数
     */
    @Insert("INSERT INTO t_user(username,password,age,email) VALUES (#{username}, #{password}, #{age}, #{email})")
    @Options(useGeneratedKeys = true, keyProperty = "id")
    int insert(User record);

    /**
     * 根据id查询数据
     *
     * @param id 主键
     * @return 数据记录
     */
    @Select("SELECT * FROM t_user WHERE id = #{id}")
    User selectByPrimaryKey(Long id);


    /**
     * 查询所有数据
     *
     * @return 数据列表
     */
    @Select("SELECT * FROM t_user")
    List<User> selectList();

    /**
```

```
     * 根据id更新数据
     *
     * @param record 更新的数据记录
     * @return 受影响的记录行数
     */
    @Update("UPDATE t_user SET username=#{username}, age=#{age}, email=#{email} WHERE id=#{id}")
    int updateByPrimaryKey(User record);


    /**
     * 根据id删除数据
     *
     * @param id 主键
     * @return 受影响的记录行数
     */
    @Delete("DELETE FROM t_user WHERE id=#{id}")
    int deleteByPrimaryKey(Long id);
}
```

第二步：编写 Service 层代码（UserService）

> 说明：业务逻辑层，注入 Mapper，调用数据库方法（无复杂业务时做透传）

```java
package top.mqxu.week05.service;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import top.mqxu.week05.entity.User;
import top.mqxu.week05.mapper.UserMapper;

import java.time.LocalDateTime;
import java.util.List;

/**
 * @author mqxu
 * @date 2026/4/2
 * @description UserService
 **/
@Service
@RequiredArgsConstructor
@Slf4j
public class UserService {
    private final UserMapper userMapper;

    public int add(User user) {
        int result = userMapper.insert(user);
        user.setCreateTime(LocalDateTime.now());
        log.info("添加用户:{}", user);
        return result;
    }

    public User getById(Long id) {
        return userMapper.selectByPrimaryKey(id);
    }

    public List<User> list() {
        return userMapper.selectList();
    }

    public int update(User user) {
        return userMapper.updateByPrimaryKey(user);
    }

    public int delete(Long id) {
        return userMapper.deleteByPrimaryKey(id);
    }
}
```

第三步：编写 Controller 层代码（UserController）

> 说明：接口层，对外提供 RESTful 接口，接收参数、调用 Service

```java
package top.mqxu.week05.controller;

import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.*;
import top.mqxu.week05.common.Result;
import top.mqxu.week05.entity.User;
import top.mqxu.week05.service.UserService;

import java.util.List;

/**
 * @author mqxu
 * @date 2026/4/2
 * @description UserController
 **/
@RestController
@RequestMapping("/api/user")
@RequiredArgsConstructor
public class UserController {
    private final UserService userService;

    @PostMapping
    public Result<String> add(@RequestBody User user) {
        int row = userService.add(user);
        if (row != 1) {
            return Result.error("添加失败");
        }
        return Result.success("添加成功");
    }

    @GetMapping("/{id}")
    public Result<User> get(@PathVariable Long id) {
        return Result.success(userService.getById(id));
    }

    @GetMapping("/list")
    public Result<List<User>> list() {
        return Result.success(userService.list());
    }

    @PutMapping
    public Result<String> update(@RequestBody User user) {
        int row = userService.update(user);
        if (row != 1) {
            return Result.error("更新失败");
```

```java
        }
        return Result.success("更新成功");
    }

    @DeleteMapping("/{id}")
    public Result<String> delete(@PathVariable Long id) {
        int row = userService.delete(id);
        if (row != 1) {
            return Result.error("删除失败");
        }
        return Result.success("删除成功");
    }

}
```

第四步：接口测试

- 新增：POST http://localhost:8080/api/user → 请求体（JSON），id 自增、创建时间为系统时间

```json
{
  "username": "test",
  "password": "123456",
  "age": 30,
  "email": "test@qq.com",
}
```

- 查询全部：GET http://localhost:8080/api/user/list
- 根据 ID 查询：GET http://localhost:8080/api/user/1
- 修改：PUT http://localhost:8080/api/user → 请求体（JSON），id 必传

```json
{
  "id": 1,
  "username": "test1",
  "password": "654321",
  "age": 40,
  "email": "test1@qq.com",
}
```

- 删除：DELETE http://localhost:8080/api/user/1
- 注意：启动项目前确保 MySQL 服务正常，配置文件用户名/密码正确

课时 2：MyBatis XML + 动态SQL

学习目标

1. 掌握 MyBatis XML 映射文件的编写规范
2. 会用动态条件查询
3. 区分注解和 XML 的适用场景，做到按需选择
4. 掌握 XML 映射文件的路径配置

为什么需要 XML 映射?

- 注解的局限性：复杂 SQL/动态SQL /多表关联时，注解编写繁琐、可读性差
- XML 的优势：SQL 与代码分离、易维护、支持动态 SQL/SQL 复用/多表关联
- 企业规范：**简单操作用注解，复杂操作用 XML**

课堂练习：MyBatis XML 方式实现动态 SQL

第一步：配置 XML 映射文件路径

> 说明：在 application.yml 中添加 MyBatis XML 文件扫描路径，指定 resources/mapper 目录（需手动创建 ）

```yaml
mybatis:
  mapper-locations: classpath:mapper/*.xml  # 新增：扫描 resources/mapper 目录下所有 XML
  configuration:
    map-underscore-to-camel-case: true
```

第二步：UserMapper 新增方法

> 说明：只定义方法，不写任何注解实现，方法名与其对应的 UserMapper.xml 中的 ID 要对应

```java
@Mapper
public interface UserMapper {
    /**
     * 动态条件查询：用户名模糊查询 + 年龄大于等于
     *
     * @param username 用户名
     * @param minAge   最小年龄
     * @return 数据列表
     */
    List<User> selectByCondition(String username, Integer minAge);
}
```

XML 映射文件基础规范

1. 文件位置：resources/mapper/XXXMapper.xml（与接口名一致）
2. 根标签：namespace属性=接口全类名（必须一致）
3. 操作标签：id属性=接口方法名（必须一致）
4. 返回值：标签的 resultType=返回实体类全类名
5. 参数：#{参数名}，与接口方法参数名一致

第三步：编写 XML 映射文件（UserMapper.xml）

> 重点：动态 SQL 的智能拼接逻辑

- 代码

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<!-- namespace=接口全类名 -->
<mapper namespace="top.mqxu.week05.mapper.UserMapper">
  <!-- id=接口方法名，resultType=返回实体类 -->
  <select id="selectByCondition" resultType="top.mqxu.week05.entity.User">
    SELECT * FROM t_user
    <where>
      <!-- 条件1：用户名不为空时，模糊查询 -->
      <if test="username != null and username != ''">
        AND username LIKE CONCAT('%', #{username}, '%')
      </if>
      <!-- 条件2：年龄不为空时，大于等于该年龄 -->
      <if test="minAge != null">
        AND age >= #{minAge}
      </if>
    </where>
  </select>
</mapper>
```

动态 SQL 的核心标签

- 满足条件时，才拼接标签内的 SQL 语句
- 智能拼接 WHERE 子句，核心优势：
  a. 若内部有满足条件的，自动添加 WHERE 关键字
  b. 自动去掉第一个条件前的多余 AND/OR（无需手动处理）
  c. 若内部无满足条件的，则不拼接 WHERE 关键字（避免 SQL 语法错误）

第四步：编写 Service + Controller 层代码

- Service 层（UserService）

```java
@Service
@RequiredArgsConstructor
public class UserService {
    private final UserMapper userMapper;

    // 其他代码

    public List<User> search(String username, Integer minAge) {
        return userMapper.selectByCondition(username, minAge);
    }
}
```

- Controller 层（UserController）

```java
@RestController
@RequestMapping("/api/user")
@RequiredArgsConstructor
public class UserController {
    private final UserService userService;

    // 其他代码

    /**
     * 搜索功能
     *
     * @param username 用户名
     * @param minAge   最小年龄
     * @return 数据列表
     */
    @GetMapping("/search")
    public List<User> search(
        @RequestParam(required = false) String username,
        @RequestParam(required = false) Integer minAge) {
        return userService.search(username, minAge);
    }
}
```

第五步：动态条件查询测试

- 测试1：只查用户名包含“张” → GET http://localhost:8080/api/user/search?username=

张

- 测试2：只查年龄 ≥ 20 → GET http://localhost:8080/api/user/search?minAge=20
- 测试3：用户名包含“李 ”+ 年龄 ≥ 25 → GET http://localhost:8080/api/user/search?username=李&minAge=25
- 测试4：无参数（查询全部） → GET http://localhost:8080/api/user/search
- 结果：所有请求均无 SQL 语法错误，按传入参数动态拼接条件

课时 3：MyBatis-Plus 极速开发

学习目标

1. 掌握 Spring Boot3 整合 MyBatis-Plus 的核心步骤
2. 会用 BaseMapper 实现无 SQL 的单表 CRUD
3. 掌握 LambdaQueryWrapper 条件构造器的使用
4. 会用 MyBatis-Plus 实现分页查询

什么是 MyBatis-Plus?

- 定义：MyBatis 的**增强工具**，在 MyBatis 基础上做了封装，不改变原有用法
- 核心优势：**单表操作无需编写 SQL**，内置通用 CRUD/分页/条件构造，开发效率提升 80%
- 定位：单表开发的“效率神器”，与 MyBatis 无缝兼容（可混合使用）
- 官网：https://baomidou.com/

课堂练习：MP 极速开发

第一步：引入 MyBatis-Plus 依赖

> 说明：在原有 pom.xml 中添加 MP 启动器，版本 3.5.7

```xml
<properties>
  <mybatis-plus.version>3.5.7</mybatis-plus.version>
</properties>

<dependency>
  <groupId>com.baomidou</groupId>
  <artifactId>mybatis-plus-spring-boot3-starter</artifactId>
  <version>${mybatis-plus.version}</version>
</dependency>
```

第二步：编写 MP 版 Mapper（UserMPMapper）

> 说明：继承 BaseMapper，T 为实体类，**无需编写任何方法**，内置全套 CRUD

```java
package top.mqxu.week05.mapper;

import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import org.apache.ibatis.annotations.Mapper;
import top.mqxu.week05.entity.User;

/**
 * @author mqxu
 * @date 2026/4/2
 * @description UserMPMapper: 空接口：BaseMapper 已封装所有单表 CRUD 方法，无需手动编写
 **/
@Mapper
public interface UserMPMapper extends BaseMapper<User> {
}
```

BaseMapper 内置通用 CRUD方法（常用）

方法名	作用	返回值
insert(T entity)	新增单条数据	int（受影响行数）
deleteById(Serializable id)	根据ID删除	int
updateById(T entity)	根据ID修改	int

selectById(Serializable id)	根据ID查询	T（实体类）
selectList(Wrapper queryWrapper)	条件查询列表	List
selectPage(Page page, Wrapper queryWrapper)	条件分页查询	Page

第三步：实现条件 + 分页查询（Service层）

> 重点： LambdaQueryWrapper（条件构造）+ Page（分页参数）

```java
package top.mqxu.week05.service;

import com.baomidou.mybatisplus.core.conditions.query.LambdaQueryWrapper;
import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import top.mqxu.week05.entity.User;
import top.mqxu.week05.mapper.UserMPMapper;

/**
 * @author mqxu
 * @date 2026/4/2
 * @description UserMPService
 **/
@Service
@RequiredArgsConstructor
public class UserMPService {
    private final UserMPMapper userMPMapper;

    /**
     * 分页查询
     *
     * @param username 用户名
     * @param pageNum  页码
     * @param pageSize 每页条数
     * @return 分页数据
     */
    public Page<User> page(String username, Integer pageNum, Integer pageSize) {
        // 1. 构造分页参数：pageNum（页码，从1开始）、pageSize（每页条数）
        Page<User> page = Page.of(pageNum, pageSize);
        // 2. 构造条件构造器：LambdaQueryWrapper（类型安全，避免字段名写错）
        LambdaQueryWrapper<User> wrapper = new LambdaQueryWrapper<>();
        // 条件：用户名不为空时，模糊查询（第一个参数为条件，满足才拼接）
        wrapper.like(username != null && !username.isEmpty(), User::getUsername, username);
        // 3. 调用BaseMapper的selectPage方法，实现条件分页
        return userMPMapper.selectPage(page, wrapper);
    }

    /**
     * 直接调用 BaseMapper 的新增方法（无SQL）
     *
     * @param user 用户
     * @return 影响条数
```

```
     */
    public int add(User user) {
        return userMPMapper.insert(user);
    }

    /**
     * 直接调用 BaseMapper 的删除方法（无SQL）
     *
     * @param id 主键
     * @return 影响条数
     */
    public int delete(Long id) {
        return userMPMapper.deleteById(id);
    }
}
```

第四步：编写分页查询接口（Controller 层）

> 说明：分页参数设置默认值，接口返回 Page 对象（包含分页信息和数据）

```java
package top.mqxu.week05.controller;

import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.*;
import top.mqxu.week05.common.Result;
import top.mqxu.week05.entity.User;
import top.mqxu.week05.service.UserMPService;

/**
 * @author mqxu
 * @date 2026/4/2
 * @description UserMPController
 **/
@RestController
@RequestMapping("/api/user/mp")
@RequiredArgsConstructor
public class UserMPController {
    private final UserMPService userMPService;

    /**
     * 条件分页查询接口，分页参数默认：第1页，每页10条
     *
     * @param pageNum  页码
     * @param pageSize 每页数量
     * @param username 用户名
     * @return 分页数据
     */
    @GetMapping("/page")
    public Result<Page<User>> page(
        @RequestParam(defaultValue = "1") Integer pageNum,
        @RequestParam(defaultValue = "10") Integer pageSize,
        @RequestParam(required = false) String username) {
        return Result.success(userMPService.page(username, pageNum, pageSize));
    }


    /**
     * 新增接口（无SQL）
     *
     * @param user 新增数据
     * @return 新增结果
     */
    @PostMapping
```

```java
    public Result<String> add(@RequestBody User user) {
        int row = userMPService.add(user);
        if (row <= 0) {
            return Result.error("MP 新增失败");
        }
        return Result.success("MP 新增成功");
    }

    /**
     * 删除接口（无SQL）
     *
     * @param id 删除id
     * @return 删除结果
     */
    @DeleteMapping("/{id}")
    public Result<String> delete(@PathVariable Long id) {
        int row = userMPService.delete(id);
        if (row <= 0) {
            return Result.error("MP 删除失败");
        }
        return Result.success("MP 删除成功");
    }
}
```

第五步：配置 MP

编写 MP 配置类，配置 MyBatis-Plus 拦截器，添加分页插件，指定数据库类型为 MySQL。

Java

```java
package top.mqxu.week05.config;

import com.baomidou.mybatisplus.annotation.DbType;
import com.baomidou.mybatisplus.extension.plugins.MybatisPlusInterceptor;
import com.baomidou.mybatisplus.extension.plugins.inner.PaginationInnerInterceptor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

/**
 * MyBatis-Plus 配置类
 *
 * @author mqxu
 * @date 2026/4/2
 */
@Configuration
public class MyBatisPlusConfig {

    /**
     * 配置 MyBatis-Plus 拦截器
     * 添加分页插件，指定数据库类型为 MySQL
     */
    @Bean
    public MybatisPlusInterceptor mybatisPlusInterceptor() {
        MybatisPlusInterceptor interceptor = new MybatisPlusInterceptor();
        // 添加分页插件
        // DbType.MYSQL：指定数据库类型为 MySQL，生成的分页 SQL 会使用 LIMIT
        PaginationInnerInterceptor paginationInterceptor = new PaginationInnerInterceptor(DbType.MYSQL);

        // 设置请求的页面大于最大页后的操作：true 返回首页，false 继续请求（默认 false）
        paginationInterceptor.setOverflow(false);

        // 设置单页分页条数限制（默认 500 条，-1 不限制）
        paginationInterceptor.setMaxLimit(500L);

        // 添加拦截器
        interceptor.addInnerInterceptor(paginationInterceptor);

        return interceptor;
    }
}

```

第六步：MP 分页查询测试

- 测试1：默认分页（第1页，每页10条，无条件） → GET http://localhost:8080/api/user/mp/page
- 测试2：条件分页（用户名包含“test”，第1页，每页5条） → GET http://localhost:8080/api/user/mp/page?username=test&pageNum=1&pageSize=5
- 分页返回结果说明（JSON）：

```JSON
{
  "records": [{"id":1,"username":"test","age":20,...}], // 本页数据
  "total": 4,  // 总条数
  "size": 5,   // 每页条数
  "current": 1,// 当前页码
  "pages": 1   // 总页数
}
```

课时 4：事务管理 + 项目分层规范

学习目标

1. 理解事务的核心作用（ACID），掌握企业事务使用场景
2. 会用 @Transactional 实现 Spring Boot 声明式事务
3. 掌握事务回滚的测试方法，避免事务失效
4. 熟记 Spring Boot 数据库开发的企业级规范

什么是事务？为什么需要事务？

- 事务定义：一组数据库操作，**要么全部成功，要么全部失败**，保证数据一致性
- 核心特性（ACID）：原子性（不可分割）、一致性（数据完整）、隔离性（互不干扰）、持久性（永久生效）

- 企业必用场景：转账、扣库存、多表同时操作、批量新增/修改（一步出错，全部回滚）

Spring Boot 声明式事务—@Transactional

- 核心优势：**无侵入式**，只需添加注解，无需手动编写事务开启/提交/回滚代码
- 基本用法：直接添加在**业务方法上**（Service层），适用于需要事务的方法
- 核心机制：方法执行无异常→自动提交事务；方法抛出**运行时异常**→自动回滚事务

课堂练习：带事务的业务方法

第一步：编写带事务的业务方法（UserTxService）

> 说明：模拟“新增两个用户”的业务，一步出错则全部回滚，重点理解 @Transactional 注解

```java
package top.mqxu.week05.service;

import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import top.mqxu.week05.entity.User;
import top.mqxu.week05.mapper.UserMapper;

/**
 * @author mqxu
 * @date 2026/4/2
 * @description UserTxService
 **/
@Service
@RequiredArgsConstructor
public class UserTxService {
    private final UserMapper userMapper;


    /**
     * 新增两个用户，添加工厂事务：方法内任意一步出错，全部回滚
     *
     * @param user1 用户1
     * @param user2 用户2
     */
    @Transactional
    public void addTwoUsers(User user1, User user2) {
        // 第一步：新增用户1
        userMapper.insert(user1);
        // 模拟异常：如果用户2的用户名为空，抛出运行时异常，触发事务回滚
        if (user2.getUsername() == null || user2.getUsername().isEmpty()) {
            throw new RuntimeException("用户2姓名不能为空，事务回滚");
        }
        // 第二步：新增用户2（若上面抛出异常，此步骤不会执行，用户1也会被回滚）
        userMapper.insert(user2);
    }
}
```

第二步：编写事务测试接口（UserTxController）

> 说明：提供接口调用带事务的方法，通过传参模拟异常，测试事务回滚

```java
package top.mqxu.week05.controller;

import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import top.mqxu.week05.common.Result;
import top.mqxu.week05.entity.User;
import top.mqxu.week05.service.UserTxService;

import java.util.Map;

/**
 * @author mqxu
 * @date 2026/4/2
 * @description UserTxController 事务测试接口
 **/
@RestController
@RequestMapping("/api/user/tx")
@RequiredArgsConstructor
public class UserTxController {
    private final UserTxService userTxService;

    /**
     * 新增两个用户，添加事务：方法内任意一步出错，全部回滚
     *
     * @param map 用户数据
     * @return 新增结果
     */
    @PostMapping("/addTwo")
    public Result<String> addTwo(@RequestBody Map<String, User> map) {
        User user1 = map.get("user1");
        User user2 = map.get("user2");
        userTxService.addTwoUsers(user1, user2);
        return Result.success("两个用户均新增成功");
    }
}
```

第三步：事务回滚验证

测试1：正常情况（无异常，事务提交）

- 请求：POST http://localhost:8080/api/user/tx/addTwo
- 请求体参数

```JSON
{
    "user1": {
        "username": "tx1",
        "password": "111",
        "age": 21
    },
    "user2": {
       "username": "tx2",
        "password": "222",
        "age": 22
    }
}
```

- 结果：数据库中成功新增两个用户，事务正常提交

测试2：异常情况（有异常，事务回滚）

- 请求：POST http://localhost:8080/api/user/tx/addTwo
- 请求体参数（user2 用户名为空）

```JSON
{
    "user1": {
        "username": "tx3",
        "password": "333",
        "age": 23
    },
    "user2": {
       "username": "",
        "password": "444",
        "age": 24
    }
}
```

- 结果：接口抛出异常，数据库中无新增用户，**用户 tx3 被回滚**，事务生效

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/user/tx/addTwo' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "user1": {
        "username": "tx3",
        "password": "333",
        "age": 23
    },
    "user2": {
       "username": "",
        "password": "444",
        "age": 24
    }
}'
```

Request URL

```
http://localhost:8080/api/user/tx/addTwo
```

Server response

Code	Details
500 *Undocumented*	Error: response status is 500

Response body


```
{
  "timestamp": "2026-04-02T10:00:23.855+00:00",
  "status": 500,
  "error": "Internal Server Error",
  "trace": "java.lang.RuntimeException: 用户2姓名不能为空，事务回滚\r\n\tat top.mqxu.week05.service.UserTxService.addTwoUsers(UserTxService.java:32)\n\tat java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)\n\tat java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:77)\n\tat java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)\n\tat java.base/java.lang.reflect.Method.invoke(Method.java:569)\n\tat org.springframework.aop.support.AopUtils.invokeJoinpointUsingReflection(AopUtils.java:360)\n\tat org.springframework.aop.framework.ReflectiveMethodInvocation.invokeJoinpoint(ReflectiveMethodInvocation.java:196)\n\tat org.springframework.aop.framework.ReflectiveMethodInvocation.proceed(ReflectiveMethodInvocation.java:163)\n\tat org.springframework.transaction.interceptor.TransactionAspectSupport.invokeWithinTransaction(TransactionAspectSupport.java:380)\n\tat org.springframework.transaction.interceptor.TransactionInterceptor.invoke(TransactionInterceptor.java:119)\n\tat org.springframework.aop.framework.ReflectiveMethodInvocation.proceed(ReflectiveMethodInvocation.java:184)\n\tat org.springframework.aop.framework.CglibAopProxy$DynamicAdvisedInterceptor.intercept(CglibAopProxy.java:728)\n\tat top.mqxu.week05.service.UserTxService$$SpringCGLIB$$0.addTwoUsers(<generated>)\n\tat top.mqxu.week05.controller.UserTxController.addTwo(UserTxController.java:35)\n\tat java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)\n\tat java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:77)\n\tat java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)\n\tat java.base/java.lang.reflect.Method.invoke(Method.java:569)\n\tat org.springframework.web.method.support.InvocableHandlerMethod.doInvoke(InvocableHandlerMethod.java:258)\n\tat org.springframework.web.method.support.InvocableHandlerMethod.invokeForRequest(InvocableHandlerMethod.java:191)\n\tat org.springframework.web.servlet.mvc.method.annotation.ServletI
```


企业级开发规范

1. 技术选择规范

- 简单单表 CRUD：MyBatis 纯注解（开发快）
- 复杂 SQL/动态SQL/多表关联：MyBatis XML（易维护）
- 单表快速开发/分页/条件查询：MyBatis-Plus（提效）
- 多步数据库操作：必须添加 @Transactional（保数据一致）

2. 分层开发规范

- Controller 层：仅做接口暴露、参数接收、结果返回，**不写业务逻辑**
- Service 层：核心业务逻辑处理、事务控制、调用 Mapper 层，**不直接操作数据库**
- Mapper 层：仅做数据库访问，**不写业务逻辑**，注解/XML 按需选择

3. 通用规范

- 参数传递：使用 #{参数}，防止 SQL 注入

- 注入方式：使用 @RequiredArgsConstructor（构造器注入），替代 @Autowired
- 命名规范：表名/字段名下划线，实体类/方法名驼峰，Mapper 接口名 XXXMapper
- 异常处理：业务层抛出异常，控制层统一捕获

课程总结 + 常见问题

核心知识点总结

1. 环境配置：Spring Boot 3.5.11 + JDK 17 + MySQL 8，开启驼峰转换
2. MyBatis 注解：@Mapper + CRUD 注解，简单单表操作首选
3. MyBatis XML：复杂操作用动态 SQL
4. MyBatis-Plus：BaseMapper\LambdaQueryWrapper\Page，单表开发提效神器
5. 事务管理：@Transactional 添加在 Service方法 上，保证数据一致性
6. 开发规范：分层开发、按需选技术、防 SQL 注入、规范命名

常见问题答疑

1. Q：为什么启动项目报错“找不到Mapper”?

   A：检查是否加了 @Mapper 注解，或是否配置了 @MapperScan 扫描包
2. Q：为什么数据库字段和实体类字段映射不上?

   A：检查是否开启了驼峰转换，或字段名/属性名是否对应
3. Q：为什么事务不生效?

   A：① 注解加在了 Controller层（必须加在 Service 层）；② 方法抛出了检查型异常（@Transactional 默认只回滚运行时异常）；③ 方法被 private 修饰（注解对私有方法无效）
4. Q：MyBatis 和 MyBatis-Plus 能混合使用吗?

   A：可以，MP 是 MyBatis 的增强，可在同一个项目中同时使用注解/XML/MP